

Testing

All tests can be found under the `src/test` directory. The tests files can be categorized into three different categories. These are unit test files, an integration test file, and two utility files for testing.

Unit Tests

These tests are located in the subdirectories of `src/test/java/org/troy/capstone/`. They test individual methods and classes in isolation from most of the program. Tests have been designed to target most methods individually, with test cases being used to go after each branch of each method. Many tests make use of the `ParameterizedTest` annotation to allow multiple cases to run for each test written into code.

Integration Tests

There is one integration test file, `src/test/java/org/troy/capstone/MainTest.java`. This file tests the `SearchEngine` class, which is responsible for applying all filters and sorting to the data. This class is a good candidate for integration testing, as it relies on many different classes and methods to operate correctly.

- The first test `testDisplayInSimilarItemsAndRecentlyViewedItems` clicks on an item in the `SearchedItemPagination` and checks that it gets displayed in the `RecentlyViewedWindow` as well as checking that the similar items are correctly displayed in the `SimilarItemsContainer`. A video of the test running can be seen [HERE](#).
- The second test `testFilteredSearch` makes a selection for each filter type and checks that the right number of results are returned, and that the `SearchedItemPagination` is updated with the correct number of pages. A video of the test running can be seen [HERE](#).

Utility Files for Testing

- `TestUtils.java` contains three utility methods for testing.

- ```
public static <T extends Node> T lookupByTestId(TestFxId testFxId)
```

- This returns any `Node` or descendant of `Node` in the scene graph that matches the given `TestId`.

- ```
public static <T extends Node> T lookupByTestId(String testFxId)
```

- This does the same thing as the other `lookupByTestId` method, but it takes in a `String`, since the ID for some nodes are a template and then the full ID is not known until runtime.

- ```
public static boolean equals(Table table1, Table table2)
```

- This checks if two `Table` objects are equal by checking that they have the same number of columns, the same number of rows, the same column names in the same order, and the same values in every column of every row.

## Coverage

- Test coverage is calculated with the Jacoco reporting plugin and can be found in the `generated_docs/coverage/` directory. To properly view the report open the `index.html` file in that directory in a web browser once cloned (or view it in GitHub Pages). This detailed report includes coverage for methods, lines, and branches for each class in the project. The tests have been designed to give as much coverage as possible.

## Missed coverage

- Instructions - 18/6365 missed (0.28% missed)
- Branches - 0/263 missed (0% missed)
- Cyclomatic Complexity - 0/403 missed (0% missed)
- Lines - 6/1297 missed (0.46% missed)
- Methods - 0/262 missed (0% missed)
- Classes 0/47 missed (0% missed)
- The only code missed was the catch statement in the following snippet as well as two other identical snippets in `AttributedImageContainer`, except one is for the author name and the other is for the source name.

```
imageView.setOnMouseClicked(e -> {
 try {
 desktop.browse(new URI(item.getImageUrl()));
 } catch (IOException | URISyntaxException ex) {
 System.err.println("Failed to open image URL: " +
item.getImageUrl());
 }
});
```

## Techniques Applied

- Reflection
  - This was heavily used to gain access to fields and methods of classes that test classes would not normally have access to. This allowed for more in depth testing without having to change access modifiers of fields and methods just for testing purposes. This permitted in depth testing without having to change the design of the code just for testing purposes, which is a good practice to follow.
- Mocking
  - As this program integrates its components a great deal, testing individual components sometimes requires the instantiation of other components, even if they are not the focus of the test. In order to avoid having to use the full implementation of these other components, Mockito mocking was used to create mock versions of these components that could be used in testing.

This allowed for more focused testing on the component being tested, without having to worry about the implementation details of other components that are not the focus of the test.

- Mockito enabled the disabling of certain behaviors that can be problematic, such as preventing the clicking of external links in the `AttributedItemContainer` tests from opening a web browser during testing. In this case, we tested that the right link was clicked, but we prevented the link from opening a browser tab.
- Mocking also verified certain method calls, such as in the `GeneralManager` and `QueryFilter` tests, where we verified that certain methods were called the expected number of times with the expected arguments. This allowed for more thorough testing of the interactions between different components and methods.
- Analyzing Current Coverage
  - Jacoco's coverage report was used to analyze the current coverage of the tests and identify areas that were not being covered. This allowed for the creation of new test cases to target these areas and increase overall coverage. The goal was to achieve as close to 100% coverage as possible, and the Jacoco report was instrumental in identifying areas that needed more testing.
- Breaking Apart Conditionals
  - Some conditionals that combined boolean expressions with `||` and `&&` were broken into separate conditionals to achieve higher coverage. This was necessary because the coverage sees a 2 expression conditional as having 4 branches, and this makes it harder to achieve full branch coverage, especially in cases where one half checks for `null` and the other half checks for an `empty` value.
- Config
  - A file called `Config.java` is used to load in an `app.properties` file that contains a configuration for enabling/disabling building the `SimilarItemsGraph`. This is important as the process of building the `SimilarItemsGraph` is time-consuming, and it is not necessary to build it for most tests. By using this config file, we can easily enable or disable the building of the `SimilarItemsGraph` for testing purposes, which allows for faster testing when the graph is not needed. The file in the `main` directory contains an enabled value for building the graph, while the file in the `test` directory contains a disabled value for building the graph.

## Drawbacks

- Time consuming
  - The graph has to be built twice during tests, once for a unit test regarding the graph, and once for one of the integration tests that relies on the graph. This is time-consuming, but necessary to achieve full coverage of the code.
- Reflection breaking
  - Reflection is brittle as it uses `String` values to access fields and methods, which can break if the names of these fields and methods are changed. This is a negative for reflection, but is necessary to avoid changing access modifiers. Additionally, if parameters of methods are changed, this can also break reflection calls to these methods.